Attack-Aware Forensic Receipts for Accountable LLM Decoding Services

George Chidera Akor*, Love Allen Chijioke Ahakonye†, Jae Min Lee*, Dong-Seong Kim*
*IT Convergence Engineering and NSLab Co. Ltd., Kumoh National Institute of Technology, Gumi, South Korea
†ICT Convergence Research Center, Kumoh National Institute of Technology, Gumi, South Korea
Corresponding author: dskim@kumoh.ac.kr

***Abstract*—Operational audits of LLM services still provide limited direct evidence that decoding-time policy controls were enforced as declared. An operator or intermediary can modify top-k/top-p/temperature settings, replay randomness, reorder transcript fragments, or rewrite shortlist artifacts without leaving machine-verifiable evidence. We present an attack-aware forensic architecture that combines policy commitments, per-step receipts, tamper-evident chaining, and deterministic re-execution to verify transcript consistency with a declared decoding policy. Our prototype achieves sub-millisecond per-step latency (0.021–0.067 ms/step) and compact evidence artifacts (8–307 KB). Re-execution guarantees detection of policy mismatch, randomness replay, and transcript integrity attacks; candidate-list manipulation detection additionally requires ground-truth shortlists. A baseline comparison shows re-execution is necessary for reliable semantic attack detection (5/5 vs. 2/5 for the strongest alternative). We validate across 15 configurations with 0.0% false positives (0/10,000) and confirm 100% detection on real GPT-2 logit distributions across 100 diverse prompts when ground-truth shortlists are available.**

***Index Terms*—forensic audit, stochastic decoding, tamper-evident receipts, attack detection, verifiable AI services**

I. Introduction

LLM services are now used in safety filtering, moderation, compliance review, and downstream decision support [1]–[3]. Regulatory frameworks such as the EU AI Act [4] increasingly require that high-risk AI systems maintain auditable records of their decision processes. In these deployments, accountability disputes span multiple layers, including model provenance and runtime decoding behavior. Temperature scaling, top-k, and nucleus sampling directly affect user-visible outputs, but these controls are still commonly treated as implementation details rather than auditable evidence [5], [6].

This creates a security gap: a malicious insider can steer outcomes by changing sampling parameters, a relay can tamper with candidate lists, and a compromised service can replay randomness [7]. Prior work on blockchain-enhanced data falsification detection [8] and explainable intrusion detection [9] has addressed analogous tamper-detection and attribution problems in IoT and IIoT networks; we extend these concerns to the LLM decoding layer. Recent systems for verifiable ML have advanced forward-pass assurance [10]–[12], but even with authentic logits, investigators need independent evidence that the emitted transcript follows the declared policy. We therefore design a forensic audit pipeline that binds policy context, randomness context, candidate digest, selected token, and chain state at each decoding step, enabling verifiers to evaluate transcript consistency and return reason-coded outcomes. We scope guarantees to policy execution over a provided shortlist and target asynchronous verification modes [3], [13].

Contributions. (1) A decoding-specific adversarial taxonomy with security requirements derived from incident-response needs. (2) A tamper-evident receipt model binding policy, randomness, candidates, and tokens into chain-of-custody artifacts. (3) Formal detection propositions mapping four attack classes to verifier checks. (4) Validation on a SHA-256 prototype across 15 configurations (0.0% FP, sub-ms latency, 100% detection on GPT-2 logits) with a three-baseline comparison showing re-execution is necessary for semantic attack detection.

II. Threat Model and Security Requirements

A. System roles and trust boundaries

We consider five roles: client, model-serving provider, receipt generator, verifier, and evidence store. The provider and receipt generator may share one administrative domain, while verifiers may be internal auditors, external assessors, or compliance services. We assume standard cryptographic hardness for commitments and proofs, but we do not assume provider-side runtime logs are inherently trustworthy.

This work targets decoding-execution accountability. It does not substitute for assurances about model training provenance, model weights, or full forward-pass correctness.

B. Adversaries

We model four attacker classes:

- **Compromised provider component** that modifies decoding policy parameters at runtime.
- **Malicious relay/proxy** that tampers with candidate lists or output ordering in transit.
- **Insider attacker** with privileged log access who attempts post-hoc evidence rewriting.
- **Faulty automation** that unintentionally reuses randomness context or truncates transcripts.

C. Assets and guarantees

The primary assets are policy integrity, randomness integrity, candidate-list integrity, transcript continuity, and evidence chain-of-custody. Within the decoding layer, the system

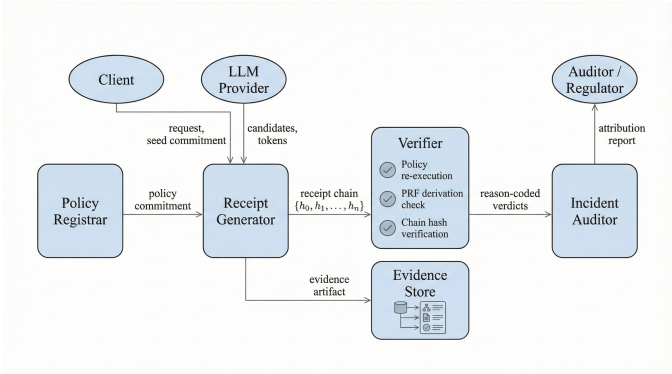

Fig. 1. Forensic audit architecture. The receipt generator binds policy, randomness, and candidate data into a hash-chained receipt at each decoding step. The verifier re-executes decoding under committed parameters and emits reason-coded verdicts for the incident auditor.

provides three guarantees: (i) consistency between declared policy and sampled tokens over the provided shortlist; (ii) tamper evidence for transcript edits; and (iii) failure localization signals for incident triage.

III. Forensic Audit Architecture

A. Module overview

The architecture contains five modules: policy registrar, receipt generator, verifier, evidence store, and incident auditor (Fig. 1). The modules integrate with existing provenance and logging pipelines rather than replace them.

- **Policy registrar:** records versioned decoding-policy commitments and authorized update windows.
- **Receipt generator:** emits per-step forensic receipts bound to request context.
- **Verifier:** checks receipt-chain validity and policy-consistent decoding transitions for committed shortlist artifacts.
- **Evidence store:** preserves receipts and proofs as immutable incident artifacts.
- **Incident auditor:** converts verifier outcomes into attribution reports and remediation actions.

B. Receipt structure and chain update

At decoding step t, the system records a tuple containing request identifier, policy commitment, randomness binding, candidate digest, selected token, step index, and previous-chain hash. The chain state is updated as:

$$h_t = H\big(\mathtt{dom} \| h_{t-1} \| \mathtt{rid} \| \mathtt{ph} \| \mathtt{sc} \| t \| \mathtt{ch}_t \| y_t \| W_t^s \| R_t\big) \quad (1)$$

where H is SHA-256, $\mathtt{dom}$ is a domain separator, h_{t-1} the previous chain hash, $\mathtt{rid}$ the request identifier, $\mathtt{ph}$ the policy hash, $\mathtt{sc}$ the seed commitment, $\mathtt{ch}_t$ the candidate-set hash, y_t the selected token, W_t^s the sampling weight, and R_t the random draw. Any change propagates to a chain mismatch.

Algorithm 1 Transcript Verification

Require: Transcript $\mathcal{T} = \{(y_t, W_t^s, R_t, \mathtt{ch}_t, h_t)\}_{t=0}^{N-1}$, policy P, seed σ, request id $\mathtt{rid}$
Ensure: Verdict v, failure set $\mathcal{F}$

1: $\mathcal{F} \leftarrow \emptyset$; $\hat{h} \leftarrow H(\mathtt{dom}_0 \| \mathtt{rid} \| \mathtt{ph} \| \mathtt{sc})$ {init receipt}
2: **for** $t = 0$ **to** $N-1$ **do**
3: $\quad U_t \leftarrow \mathrm{PRF}(\sigma, t)$ {re-derive randomness}
4: $\quad (\hat{y}_t, \hat{W}_t^s, \hat{R}_t) \leftarrow \textsc{DecodeStep}(P, \mathtt{cands}_t, U_t)$
5: $\quad$ **if** $\hat{y}_t \neq y_t$ **or** $\hat{W}_t^s \neq W_t^s$ **or** $\hat{R}_t \neq R_t$ **then**
6: $\quad\quad \mathcal{F} \leftarrow \mathcal{F} \cup \{(t, \textsc{PolicyOrRandMismatch})\}$
7: $\quad$ **end if**
8: $\quad \hat{h} \leftarrow H(\mathtt{dom} \| \hat{h} \| \mathtt{rid} \| \mathtt{ph} \| \mathtt{sc} \| t \| \mathtt{ch}_t \| y_t \| W_t^s \| R_t)$
9: $\quad$ **if** $\hat{h} \neq h_t$ **then**
10: $\quad\quad \mathcal{F} \leftarrow \mathcal{F} \cup \{(t, \textsc{ChainMismatch})\}$
11: $\quad$ **end if**
12: **end for**
13: $v \leftarrow (\mathcal{F} = \emptyset \ ?\ \textsc{Pass} : \textsc{Fail})$
14: **return** $(v, \mathcal{F})$

C. Verification procedure

Algorithm 1 formalizes the verification logic. The verifier deterministically re-executes each decoding step under the committed policy, recomputes the chain hash, and emits reason-coded failures for triage.

IV. Attack Scenarios and Detection Logic

We analyze four tampering scenarios detectable from decoding receipts. For each, we state a detection proposition and provide a proof sketch reducing to standard cryptographic assumptions (collision resistance of H, pseudorandomness of PRF, determinism of DecodeStep). All proofs assume that DecodeStep is a deterministic function of its inputs and that H is collision-resistant.

A. Policy mismatch attack

An attacker changes temperature or top-p parameters after policy registration. Detection is triggered when recomputed transitions under committed policy fail to match receipt-declared transitions.

Proposition 1. *For any transcript produced with altered policy parameters $P' \neq P$, the decoding re-execution check detects the mismatch at every step where* $\textsc{DecodeStep}(P', \mathtt{cands}_t, U_t) \neq \textsc{DecodeStep}(P, \mathtt{cands}_t, U_t)$, *assuming* DecodeStep *is deterministic and H is collision-resistant.*

Proof sketch. Since DecodeStep is deterministic, $P' \neq P$ causes a re-execution mismatch at every step where the policy difference affects the cumulative distribution (Algorithm 1, line 5). In the degenerate case where outputs coincide despite $P' \neq P$, two sub-cases arise: (a) if the attacker writes $\mathtt{ph}' = H(P')$ into the receipt, the chain-hash check catches the mismatch because $H(P') \neq H(P)$ under collision resistance; (b) if the attacker writes the committed $\mathtt{ph} = H(P)$ and all step outputs are identical to those under P, the attack evades

detection entirely, but harmlessly, since the emitted transcript is identical to honest execution. □

This guarantee holds for all perturbations that affect the cumulative distribution: our experiments confirm detection when δ_T is as small as 0.0015% of T. However, an adaptive adversary can identify *degenerate steps* where $P' \neq P$ produces identical (y, W_t^s, R_t), e.g., when one candidate has overwhelming probability. At such steps, the policy change is undetectable (100% evasion at degenerate entropy, 40.9% at medium entropy in our experiments). This is harmless: the adversary gains no utility because the output is identical to honest execution. Any perturbation that changes the output is detected with probability 1 (Section VI).

B. Randomness replay or bias attack

An attacker reuses or manipulates randomness context to force predictable outcomes. Detection combines commitment checks with replay tests over nonce/request domains. Optional statistical alarms flag suspicious concentration patterns for analyst review.

Proposition 2. *Any modification to the per-step random draw U_t is detected by the verifier at every step where the modified draw U'_t causes* $\textsc{DecodeStep}(P, \texttt{cands}_t, U'_t) \neq \textsc{DecodeStep}(P, \texttt{cands}_t, U_t)$*, assuming* PRF *security and collision resistance of H. Even when the outputs coincide despite $U'_t \neq U_t$, the chain-hash check detects the modification unless the attacker forges a collision.*

Proof sketch. The verifier re-derives $U_t = \mathrm{PRF}(\sigma, t)$ from the committed seed and re-executes DECODESTEP. If $U'_t \neq U_t$ changes any intermediate quantity, the re-execution check detects the difference. In the degenerate case where outputs coincide, the attacker cannot find σ' with $\mathrm{PRF}(\sigma', t) = U'_t$ for tampered steps while matching honest steps, by PRF security; committing to σ exposes $U_t \neq U'_t$ at re-derivation, while committing to σ' causes mismatches at untampered steps. □

The PRF-derivation check catches all bias attacks at any level, since the attacker must modify U_t to introduce bias. A complementary statistical heuristic (Section VI-F) provides a second detection layer when PRF commitments are unavailable.

C. Candidate-list manipulation attack

An attacker injects or removes shortlist entries before sampling. Detection occurs when candidate digests in receipts diverge from verifier-reconstructed digests or from signed shortlist snapshots.

Proposition 3. *Given ground-truth candidate sets, any substitution, injection, or removal of candidates is detected via candidate-hash mismatch, assuming H is collision-resistant.*

Proof sketch. The receipt records $\texttt{ch}_t = H(\texttt{cands}'_t)$; the verifier computes $H(\texttt{cands}_t)$ from ground truth. Since $\texttt{cands}'_t \neq \texttt{cands}_t$, equality of hashes constitutes a collision, which occurs with probability $\leq 2^{-128}$ under SHA-256. Without ground-truth candidates this check is inoperative. □

Without ground-truth candidates, detection of logit-proximate substitutions (within 1 logit unit) drops to 0%. With ground-truth candidates, detection is 100%. This asymmetry is a key limitation discussed in Section VII.

D. Transcript truncation/reordering attack

An attacker drops or reorders token segments before evidence handoff. Hash-chain continuity checks and step-index consistency checks expose gaps and out-of-order segments.

Proposition 4. *Any dropped or reordered step causes a chain-hash mismatch at the first affected position, with exact localization to the splice point, assuming H is collision-resistant.*

Proof sketch. A dropped step shifts all subsequent data by one position, causing the verifier to compute h_t^* over a different prior hash and step index than the recorded h_{t+1}; equality would require a collision in H. Reordering similarly changes the step-index input at the first affected position. In both cases, the mismatch propagates by induction since each h_t feeds into h_{t+1}. □

In transcript-splicing experiments (steps from two different requests), the verifier achieves 100% detection and 100% localization to the exact splice point.

V. Related Work

Verifiable ML computation. Delphi [10], ZKML [6], ZKTorch [12], and zkLLM [11] prove forward-pass correctness but do not address whether decoding-layer policy controls were applied as declared; our architecture is complementary. Watermarking [14] embeds statistical provenance signals; our approach instead provides deterministic, per-step evidence of policy compliance.

Commitment schemes and auditable AI. Prediction commitments [15] and co-piloted auditing workflows [3] make ML pipelines auditable at the model level [16]. Our receipts operate at finer granularity, binding policy, randomness, and candidate digests per decoding step.

Blockchain and verifiable randomness. On-chain provenance registries [13], VRFs [17], and scalable randomness beacons [18] provide tamper-evident anchoring and honest-randomness guarantees. Our architecture can optionally anchor receipt roots on a ledger, and our per-step U_t derivation follows a PRF pattern bound to the policy commitment, enabling replay detection without on-chain VRF verification.

Log integrity and forensics. Hash-chain and Merkle-tree constructions [19] underpin tamper-evident logging. Our design extends beyond passive integrity: deterministic re-execution detects policy and randomness manipulations that signed logs alone cannot distinguish from legitimate behavior.

VI. Experimental Evaluation

A. Experimental setup

We implement the receipt generator and verifier in Python with SHA-256 (Table I). For scaling experiments, candidate logits are assigned by rank: the highest-ranked candidate receives logit $+2.0$ (in Q16.16 fixed-point) and the lowest

TABLE I
EXPERIMENTAL PARAMETERS.

Parameter	Value
Language	Python
Hash function	SHA-256 (domain-separated)
Candidate-set sizes K	16, 32, 64
Transcript lengths N	16, 32, 64, 128, 256
Logit distribution	Rank-weighted (see text)
Top-k	$\lfloor K/4 \rfloor$
Top-p (Q16.16)	0.9
Temperature (Q16.16)	1.0
Latency runs per config	100 (5 warmup runs discarded)
FP honest transcripts	10,000 (incl. edge cases)
Bias heuristic FP trials	1,000
GPT-2 validation prompts	100 (4 categories $\times$ 25)
GPT-2 attack types tested	5 (all 4 classes; manip. $\pm$ GT)
Platform	Intel Core i7, Ubuntu 24.04 (WSL 2)

receives -3.0, producing a peaked distribution that approximates the heavy-tailed output distributions observed in real language models. We additionally validate on logits extracted from GPT-2 (117M) across 100 prompts in four categories (e.g., "*The central bank raised interest rates by 25 basis points on*"), with $K = 32$ and $N = 32$ per prompt (3,200 total steps). All latency measurements are single-threaded. Production deployment would replace SHA-256 with Poseidon for SNARK compatibility [20]. The implementation and all experimental scripts are publicly available.[1]

B. Security analysis: baseline comparison

We compare our forensic verifier against three baselines of increasing strength: (i) a *signed Merkle log* that records token hashes in a Merkle tree with a signed root; (ii) a *policy-commitment verifier* that binds policy parameters via hash and signs per-step records with HMAC, but does not re-execute decoding; and (iii) a *watermark-based detector* that embeds a Kirchenbauer et al. [14] style statistical signal by biasing sampling toward a hash-partitioned green list and testing for the signal via a one-proportion z-test.

Table II summarizes the results across all five attack classes. The policy-commitment baseline and the Merkle baseline both detect only structural attacks; the watermark detector exhibits partial, unreliable detection because attacks that change selected tokens sometimes disrupt the green-list pattern but do not reliably do so.

The key insight is that re-execution is necessary for detecting semantic attacks. The policy-commitment baseline binds parameters but cannot verify execution consistency; the watermark baseline detects wholesale text replacement but is blind to policy-parameter changes and randomness replay that preserve the green-list pattern. Both non-re-execution baselines detect only structural attacks (drops and reorders) via step-index checking.

C. Adaptive adversary analysis

We implement an adaptive adversary with full access to the verification algorithm, testing 200 trials across four entropy levels (degenerate, low, medium, high) at $K = 16$, $N = 32$. We distinguish *policy evasion* (using $P' \neq P$ without detection) from *output evasion* (changing the emitted token). Three strategies are evaluated. (a) *Degenerate-case search:* policy evasion exists (at degenerate entropy 100% of perturbations produce identical outputs, declining to 40.9% at medium entropy), but output evasion is 0% at every entropy level, confirming that the re-execution guarantee is a hard wall with zero adversarial utility. (b) *Optimal evasion budget:* the adversary pre-screens steps and tampers only where outputs are unchanged; 100% of steps admit such a perturbation, yet no output-changing perturbation evades detection. (c) *Collision-proximate attack:* candidate-hash Hamming distance under single-token substitution averages 128.0 bits (minimum 91 bits), providing no exploitable structure for hash collision.

[1]https://github.com/Jupiter-Plantagenet/vrbdecode-icufn

TABLE II
ATTACK DETECTION CAPABILITY: FORENSIC VERIFIER VS. THREE BASELINES. "DETECTED" = $\geq$99% DETECTION RATE ACROSS ALL TESTED INTENSITIES. "PARTIAL" = DETECTED IN SOME CONFIGURATIONS BUT NOT RELIABLY. "NOT DET." = 0% DETECTION.

Attack class	Forensic verifier	Policy-commit	Water-mark	Merkle log
Policy mismatch	Detected	Not det.	Not det.	Not det.
Randomness replay	Detected	Not det.	Not det.	Not det.
Candidate manip. (GT)	Detected	Not det.	Partial	Not det.
Transcript drop	Detected	Detected	Partial	Detected
Transcript reorder	Detected	Detected	Partial	Detected
Total (reliable)	5/5	2/5	0/5	2/5

TABLE III
VERIFICATION LATENCY AND EVIDENCE SIZE (K: CANDIDATE-SET SIZE, N: TRANSCRIPT LENGTH). MEASURED ON AN INTEL CORE I7 DESKTOP, PYTHON 3.12, UBUNTU 24.04 (WSL 2), 95 MEASURED RUNS PER CONFIGURATION AFTER 5 WARMUP.

K	N	Mean (ms)	P95 (ms)	Size (KB)	Tput (t/s)	Per-step (ms)
16	16	0.35	0.40	8.4	2,899	0.022
16	64	1.51	2.15	32.3	662	0.024
16	256	5.65	8.91	128.4	177	0.022
32	16	0.59	0.69	12.1	1,686	0.037
32	64	2.37	2.95	47.3	421	0.037
32	256	9.31	10.13	188.4	107	0.036

D. Operational metrics

Table III reports verification latency, evidence size, throughput, and per-step cost across configurations. Fig. 2 visualizes the scaling behavior.

Scaling behavior. Latency scales linearly with N and sub-linearly with K (1.7$\times$ per-step cost when doubling K from 16 to 32), confirming $O(N)$ complexity with no hidden superlinear factors (Fig. 2). Evidence sizes range from 8.4 KB ($K{=}16$, $N{=}16$) to 306.8 KB ($K{=}64$, $N{=}256$).

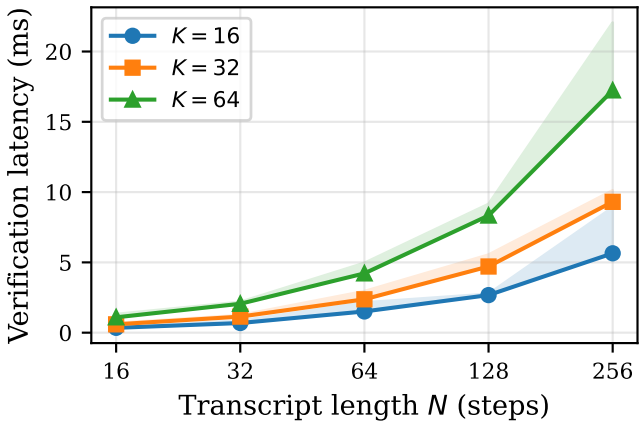

Fig. 2. Verification latency vs. transcript length N for $K \in \{16, 32, 64\}$. Shaded bands indicate P95 latency. Scaling is linear in N.

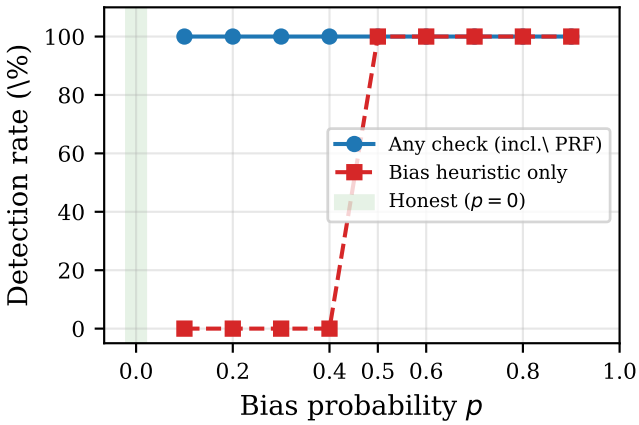

Fig. 3. Bias detection power vs. bias probability p ($K = 16$, $N = 32$). The PRF-derivation check catches all bias levels; the statistical heuristic adds a second detection layer with a step-function threshold at $p = 0.5$.

E. Correctness validation

Detection follows from Propositions 1–4: any deviation in P, U_t, or step ordering produces a mismatch, with security reducing to collision resistance and PRF pseudorandomness. Degenerate cases where outputs coincide despite parameter changes are harmless (identical to honest execution) as discussed in Proposition 1. We validate across all 15 (K, N) configurations, confirming 100% detection when at least one tampered step is present.

False-positive rate. We measure the FP rate across 10,000 honest transcripts spanning all (K, N) configurations and edge cases (near-boundary top-p, extreme temperature $T = 1$, all-ties logits, minimal $K = 1$). The observed rate is 0/10,000; a 95% Wilson confidence interval gives $[0.0\%, 0.038\%]$.

F. Bias heuristic characterization

As a complementary detection layer, we evaluate a statistical heuristic that flags transcripts where any single token ID is selected more than $\max(N/2, 8)$ times, indicating potential selection bias.

Under Zipf-like distributions modeling honest decoding, the false-positive rate is 0.0% (0/1,000 honest transcripts). Fig. 3 shows the detection characteristic: the heuristic exhibits a step function at $p = 0.5$, with 0% detection for $p < 0.5$ and 100% for $p \geq 0.5$. Overall detection (including the PRF check) is 100% at all bias levels, since the attacker must modify U_t to inject bias. This sharp transition means the heuristic alone is effective against strong bias but does not detect subtle distributional steering.

G. Validation with real LLM logits

We validate on real logit distributions extracted from GPT-2 (117M) across 100 prompts in four categories (25 per category, 3,200 total steps), testing all four attack types with 95% Wilson CIs.

Honest verification. All 100 honest transcripts pass verification (FP rate 0.0%, 95% Wilson CI $[0.0\%, 3.7\%]$).

TABLE IV
GPT-2 ATTACK DETECTION RATES (100 PROMPTS, $K = 32$, $N = 32$).

Attack type	Det.	Total	Rate (95% Wilson CI)
Policy mismatch ($T/2$)	100	100	100.0% [96.3, 100.0]
Randomness replay	100	100	100.0% [96.3, 100.0]
Candidate manip. (GT)	100	100	100.0% [96.3, 100.0]
Candidate manip. (no GT)	53	100	53.0% [43.3, 62.5]
Transcript drop (50%)	100	100	100.0% [96.3, 100.0]

Attack detection. Table IV reports detection rates. All four attack types with ground-truth shortlists achieve 100% detection ($[96.3\%, 100.0\%]$). Candidate manipulation without ground truth achieves 53% ($[43.3\%, 62.5\%]$), confirming the trust boundary in Section VII.

Entropy analysis. Real logit distributions exhibit wider per-step entropy variation than our Zipf approximation, ranging from 0.001 bits (near-deterministic code completions) to 4.95 bits (open-ended conversational), with an overall mean of 2.81. The verification pipeline handles this full entropy range without false positives.

Latency. Honest verification latency on real GPT-2 transcripts averages 1.19 ms per transcript (0.037 ms/step, 95% CI $[1.17, 1.21]$ ms), consistent with the synthetic scaling results in Table III.

VII. Limitations

The architecture has six limitations: (1) candidate-list manipulation detection requires ground-truth shortlists; (2) at degenerate steps, harmless policy evasion is possible (identical outputs under $P' \neq P$) but output-changing evasion is 0%; (3) the bias heuristic cannot detect subtle bias at $p < 0.5$; (4) SHA-256 is not ZK-friendly; production requires Poseidon; (5) floating-point non-determinism across platforms could cause spurious mismatches; (6) evidence sizes grow linearly with K and N.

VIII. Conclusion

We presented an attack-aware forensic architecture for accountable LLM decoding services. Deterministic re-execution guarantees detection of policy mismatch, randomness replay, and transcript integrity attacks; candidate-list manipulation additionally requires ground-truth shortlists. A three-baseline comparison confirms re-execution is necessary for semantic attack detection (5/5 vs. 2/5). Validation across 15 configurations yields 0.0% false positives (0/10,000), sub-millisecond per-step latency, and 100% detection on GPT-2 logits from 100 prompts with compact evidence artifacts (8–307 KB). Future work targets Poseidon-based hashing for SNARK integration, cross-platform determinism testing to characterize floating-point mismatch rates, and field evaluation in production LLM serving pipelines with real-time audit workflows.

References

[1] H. Inan, K. Upasani, J. Chi, R. Rungta, K. Iyer, Y. Mao, M. Tontchev, Q. Hu, B. Fuller, D. Testuggine, and M. Khabsa, "Llama guard: LLM-based input-output safeguard for human-AI conversations," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL 2024)*. Association for Computational Linguistics, 2024.

[2] S. Cho, T. Yun, C. Yoon, and W. Jeon, "Moderating LLMs: Challenges and guardrail strategies," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024), System Demonstrations*. Association for Computational Linguistics, 2024.

[3] H. Gu, M. Schreyer, K. Moffitt, and M. A. Vasarhelyi, "Artificial intelligence co-piloted auditing," *International Journal of Accounting Information Systems*, vol. 54, p. 100698, Sep. 2024.

[4] European Parliament and Council of the European Union, "Regulation (EU) 2024/1689 of the European Parliament and of the Council laying down harmonised rules on artificial intelligence (AI act)," Official Journal of the European Union, L series, 2024, entered into force 1 August 2024.

[5] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, "The curious case of neural text degeneration," in *Proceedings of the 8th International Conference on Learning Representations (ICLR 2020)*, 2020.

[6] B. Chen, S. Waiwitlikhit, I. Stoica, and D. Kang, "ZKML: an optimizing system for ML inference in zero-knowledge proofs," in *Proceedings of the 19th European Conference on Computer Systems (EuroSys 2024)*. ACM, 2024, pp. 560–574.

[7] J. Wei, A. Abdulrazzag, T. Zhang, A. Muursepp, and G. Saileshwar, "When speculation spills secrets: Side channels via speculative decoding in LLMs," 2024, arXiv:2411.01076.

[8] L. A. C. Ahakonye, C. I. Nwakanma, and D.-S. Kim, "Blockchain-enhanced feature engineered data falsification detection in 6G in-vehicle networks," *IEEE Internet of Things Journal*, vol. 12, no. 15, pp. 30 036–30 048, 2025.

[9] L. A. C. Ahakonye, C. I. Nwakanma, J. M. Lee, and D.-S. Kim, "Machine learning explainability for intrusion detection in the industrial Internet of Things," *IEEE Internet of Things Magazine*, vol. 7, no. 2, pp. 68–74, 2024.

[10] P. Mishra, R. Lehmkuhl, A. Srinivasan, W. Zheng, and R. A. Popa, "Delphi: A cryptographic inference service for neural networks," in *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, 2020, pp. 2505–2522.

[11] H. Sun, J. Li, and H. Zhang, "zkllm: Zero knowledge proofs for large language models," in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS 2024)*. ACM, 2024.

[12] B.-J. Chen, L. Tang, and D. Kang, "Zktorch: Compiling ML inference to zero-knowledge proofs via parallel proof accumulation," in *Proceedings of the 20th European Conference on Computer Systems (EuroSys 2025)*. ACM, 2025.

[13] G. C. Akor, L. A. C. Ahakonye, J. M. Lee, and D.-S. Kim, "ProvAuditChain: A gas-efficient on-chain provenance framework for AI-driven smart contract audits," in *Proceedings of the 16th International Conference on ICT Convergence (ICTC 2025)*. IEEE, 2025.

[14] J. Kirchenbauer, J. Geiping, Y. Wen, J. Katz, I. Miers, and T. Goldstein, "A watermark for large language models," in *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*. PMLR, 2023, pp. 17 061–17 084.

[15] B. Parno, J. Howell, C. Gentry, and M. Raykova, "Pinocchio: Nearly practical verifiable computation," in *2013 IEEE Symposium on Security and Privacy*. IEEE, 2013, pp. 238–252.

[16] S. Kapoor, R. Bommasani, K. Klyman, S. Longpre, A. Ramaswami *et al.*, "On the societal impact of open foundation models," in *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*. PMLR, 2024.

[17] S. Micali, M. Rabin, and S. Vadhan, "Verifiable random functions," in *Proceedings of the 40th Annual Symposium on Foundations of Computer Science (FOCS '99)*. IEEE, 1999, pp. 120–130.

[18] X. Meng, B. Pinkas, I. Shahaf, and K. Shwartz, "Rondo: Scalable and reconfiguration-friendly randomness beacon," in *Advances in Cryptology – ASIACRYPT 2024*. Springer, 2024.

[19] S. A. Crosby and D. S. Wallach, "Efficient data structures for tamper-evident logging," in *Proceedings of the 18th USENIX Security Symposium (USENIX Security 09)*. USENIX Association, 2009, pp. 317–334.

[20] A. Kothapalli, S. Setty, and I. Tzialla, "Nova: Recursive zero-knowledge arguments from folding schemes," in *Advances in Cryptology – CRYPTO 2022*. Springer, 2022, pp. 568–598.